Python Engineering - Programming Theory Guide

CLASS 15 - Exceptions & Debugging | Modelo mental del fallo

La programacion robusta no consiste en evitar cualquier fallo. Consiste en distinguir entre un camino normal y situaciones anormales, conservar informacion util sobre el problema y decidir conscientemente donde gestionarlo.

1. El camino normal y el camino excepcional

```
NORMAL PATH
input -> validate -> compute -> output

EXCEPTIONAL PATH
input -> problem detected -> exception -> handler or program termination
```

Una excepcion es una forma estructurada de abandonar el camino normal cuando continuar ya no tiene sentido.

2. Error no significa lo mismo que bug

Conviene separar varias ideas:

Concepto	Ejemplo	Significado
Expected external failure	Missing file	El entorno no cumple una condicion
Invalid input	NOT_A_NUMBER	Los datos violan el contrato
Bug	Wrong index / wrong algorithm	Nuestro codigo tiene un defecto
Syntax error	Invalid Python grammar	Python ni siquiera puede interpretar el codigo

No todo lo que produce una excepcion debe 'solucionarse' con except. A veces la excepcion revela un bug que debemos corregir.

3. Propagacion de excepciones

Cuando una funcion lanza una excepcion y no la captura, la excepcion vuelve a la funcion que la llamo.

```
main()
  |
  v
test_file()
  |
  v
load_trajectory_csv()
  |
  v
float('NOT_A_NUMBER')
  |
  v
ValueError
  |
  +---- propagates upward until a matching except is found
```

Eso permite detectar el problema cerca de donde ocurre y gestionarlo en una capa que sabe que respuesta ofrecer.

4. Donde detectar y donde gestionar

No siempre es el mismo sitio.

```
file_io.py
detects:
- invalid header
- wrong column count
- invalid numeric conversion

main.py
decides:
- show message
- skip this file
- continue testing next file
```

Principio: detecta donde tienes suficiente informacion para reconocer el problema; gestiona donde tienes suficiente contexto para decidir que hacer.

5. Excepciones especificas conservan significado

Archivo inexistente y dato corrupto no significan lo mismo.

```
FileNotFoundError
-> problem locating resource

ValueError
-> resource exists, but content/value is invalid
```

Si convertimos ambos en un mensaje generico 'ERROR', perdemos informacion util para diagnostico y respuesta.

6. Handling vs hiding

```
HANDLING
except ValueError as error:
    report_bad_data(error)
    continue_with_next_file()

HIDING
except:
    pass
```

Gestionar significa tomar una decision consciente. Ocultar significa hacer desaparecer la señal sin resolver ni comunicar el problema.

7. Fail fast

Si una cabecera o una fila incumple un requisito esencial, muchas veces es mas seguro detener esa carga cuanto antes.

```
wrong header
    |
    v
raise ValueError
    |
    X do not continue interpreting columns incorrectly
```

En ingenieria, un resultado numerico plausible generado a partir de datos mal interpretados puede ser mas peligroso que un fallo visible.

8. El contrato de datos

Nuestro CSV define un contrato sencillo:

```
header = h_m,v_m_s,t_s
each row = exactly 3 columns
each column = convertible to float
```

Validar el contrato en la frontera de entrada protege el resto del programa. Una vez cargado correctamente, otras funciones pueden trabajar asumiendo datos numericos consistentes.

9. raise como herramienta de diseno

`raise` no es solo una reaccion a errores. Tambien expresa nuestras propias reglas.

```
if len(values) != 3:
    raise ValueError(...)
```

Estamos traduciendo una regla de dominio/formato a una señal estandar de Python.

10. try debe ser pequeno

Cuantas mas lineas metemos dentro de try, mas operaciones diferentes pueden generar el mismo tipo de excepcion y mas dificil es saber que estamos gestionando realmente.

```
# Better mental structure
try:
    trajectory = load_trajectory_csv(path)
except ValueError:
    ...
else:
    analyze(trajectory)
```

El bloque else mantiene fuera del try el trabajo que ya no necesita proteccion frente a ese fallo concreto.

11. finally y recursos

finally expresa 'esto debe ejecutarse ocurra lo que ocurra'. Sin embargo, para recursos como archivos preferimos `with` porque encapsula especificamente el ciclo de vida del recurso.

```
with open(...) as file:
    ...
# file closed automatically
```

12. Debugging: observar estado, no adivinar

Un debugger convierte el programa en una secuencia observable. En lugar de imaginar todo el flujo mentalmente, podemos pausar y comprobar que valores existen realmente.

```
EXPECTATION
values[1] should be numeric text

OBSERVATION
values[1] == 'NOT_A_NUMBER'

CONCLUSION
float(values[1]) will raise ValueError
```

> La habilidad central no es pulsar botones del debugger; es comparar **lo que esperabas** con **lo que observas**.

13. Breakpoints como experimentos

Un breakpoint es una pregunta al programa: 'cuando llegues aqui, enseñame tu estado'.

```
breakpoint before conversion
       |
       v
inspect clean_line
inspect values
inspect line_number
       |
       v
predict next instruction
```

14. Step Into vs Step Over

Usa Step Over cuando confias temporalmente en la funcion llamada y solo te importa su resultado. Usa Step Into cuando sospechas que el problema esta dentro de esa funcion.

```
trajectory = load_trajectory_csv(path)

Step Over -> execute whole loader
Step Into -> enter loader line by line
```

15. Call Stack

El Call Stack responde: **como he llegado hasta aqui?**

```
main
 -> test_file
    -> load_trajectory_csv
       -> current line
```

> Cada nivel tiene sus propias variables locales. Esto conecta directamente con lo aprendido sobre scope.

16. Traceback: debugging despues del fallo

Si no estamos usando debugger, el traceback conserva una fotografia textual de la cadena de llamadas.

```
bottom:
ValueError: could not convert ...

above:
file_io.py line ... v = float(values[1])

above:
main.py line ... load_trajectory_csv(...)
```

Por eso recomendamos empezar abajo y subir hasta el primer punto relevante de nuestro codigo.

17. Mensajes de error como instrumentacion

Un buen mensaje reduce el tiempo de diagnostico.

```
bad:
'Invalid data'

better:
'Line 4: expected 3 columns, got 2'
```

Linea, nombre de archivo, valor problematico y expectativa suelen ser informacion util, siempre evitando mensajes innecesariamente gigantes.

18. Por que no devolver None para todo

Podriamos escribir una funcion que ante cualquier problema simplemente devolviera None. Pero entonces el caller pierde la causa concreta.

```
trajectory = load(...)

if trajectory is None:
    # Why?
    # missing file?
    # corrupt number?
    # wrong header?
    # programmer bug?
```

Las excepciones transportan informacion estructurada sobre el tipo de fallo.

19. Robustez no significa continuar siempre

A veces la respuesta correcta es continuar con el siguiente archivo. Otras veces la respuesta correcta es detener el programa. 'Robusto' significa responder de manera controlada y coherente, no ignorar todo.

Contexto	Posible respuesta
Batch de 100 archivos, uno corrupto	Reportar y continuar con los demas
Archivo critico de configuracion	Detener la aplicacion
Dato opcional	Usar valor por defecto documentado
Bug inesperado	Dejar que falle y corregir el codigo

20. Conexion con ingenieria

Sistema	Excepciones/fallos previsibles
GNSS processing	missing epoch, malformed measurement file
Test bench	missing sensor log, non-numeric field
Trajectory simulation	invalid config, impossible parameter
Optimization	solver failure / invalid domain
Data pipeline	wrong columns, corrupt record

21. Modelo mental final

```
NORMAL EXECUTION
      |
      v
operation detects invalid condition
      |
      v
EXCEPTION OBJECT
type + message + call context
      |
      v
propagates upward
      |
      +--> matching handler -> controlled response
      |
      `--> no handler -> traceback + program stops
```

Debugging: pausa, observa, compara con tu expectativa y sigue el flujo. **Exceptions:** conserva y comunica informacion cuando el flujo normal ya no es valido.

Resumen final

```
TRY      -> code that may fail in an expected way
EXCEPT   -> response to a specific exception
ELSE     -> success-only code
FINALLY  -> always-run code
RAISE    -> deliberately signal invalid state/input

GOOD PRACTICE
- specific exceptions
- small try blocks
- clear messages
- do not hide bugs
- validate data at boundaries

DEBUGGING LOOP
predict -> run/pause -> inspect -> compare -> isolate -> fix -> rerun
```